1. Data Processing

1.1 Data structure and reshaping

We start with 100 synthetic ETFs, observing `open`, `close`, and `volume` for each ETF on each day. The data arrives in **long** format (one row per ETF per day), but feature engineering requires **wide** format, matrices with rows indexed by date and columns by ETF. This reshaping enables **vectorised operations**: instead of looping over each ETF, we compute across all 100 simultaneously using matrix arithmetic (e.g. a cumsum-based rolling mean), which gives roughly a 100× speed-up over a naive nested loop.

1.4 Time Series Split and Cross-Validation Setup

Following the Week 8 tutorial we split data into training (2010-2012) and test (2013+) periods, and define rolling CV folds within training. We also define the two helper functions used throughout Section 3: `flatten_features` converts the 3D feature array to the 2D matrix LightGBM expects, and `compute_ic` measures the Information Coefficient (mean daily Spearman correlation of predicted vs actual returns).

The CV scheme is illustrated conceptually as a sliding window: each fold's training window is 252 days, the validation window is 21 days, and a gap of exactly `lookahead` days separates them — guaranteeing no target labels from the training period contaminate the validation labels.

2. Feature Engineering

Our features extend the Week 8 tutorial's feature set. We compute all 40 base features for the entire training period at once using `compute_all_features` in our strategy.

```
## Feature array dimensions [dates x ETFs x base features]: 754 100 31
```

2.1 Week 8 Baseline Features

We reuse the following technical indicators from the Week 8 tutorial: RSI (14-day relative strength), Bollinger Bands (price position vs. 20-day SMA $\pm 2\sigma$), MACD and PPO (trend indicators from 12/26-day EMA spreads), multi-horizon returns over 1/5/10/21/42/63 days, cross-sectional deciles of each return horizon across the 100 ETFs (which strip out common market moves), and date/rank features (month, weekday, and fractional rank of log dollar volume).

NATR, **ATR**, **sector**, and **sector-quantile returns** are omitted — our dataset consists of synthetic ETFs with no sector classification.

2.2 Features additional to Week 8

Beyond the Week 8 set, we add: the **MACD histogram** MACD_hist = MACD − Signal with $\text{Signal}_t = \text{Signal}_{t-1} + \frac{2}{10}(\text{MACD}_t - \text{Signal}_{t-1})$, capturing momentum acceleration; **price-to-SMA spreads** $(\text{close} - \text{SMA}_K)/\text{close}$ for $K \in \{5, 10, 20, 50, 200\}$ and the **crossover signals** $(\text{SMA}_5 - \text{SMA}_{20})/\text{close}$ and $(\text{SMA}_{10} - \text{SMA}_{50})/\text{close}$, measuring distance from trend and short-vs-long term alignment; **realised volatility** $\text{rvol}_w = \sqrt{\widehat{\text{Var}}(\log(\text{cl}_t/\text{cl}_{t-1}))_{w\text{-day}} \times 252}$ for $w \in \{5, 10, 21, 63\}$; **volume features** $\text{vol_ratio} = \text{vol}/\text{SMA}_{20}(\text{vol})$, $\text{vol_trend} = (\text{SMA}_5(\text{vol}) - \text{SMA}_{20}(\text{vol}))/\text{SMA}_{20}(\text{vol})$, and $\text{log_dollar_vol} = \log(1 + \text{close} \times \text{vol})$; the **open-close return** $(\text{close} - \text{open})/\text{open}$; **z-scores** $(\text{close} - \text{SMA}_K)/\sigma_K$ for $K \in \{20, 50\}$; and **rate of change** $\text{roc}_k = r_k \times 100$ for $k \in \{5, 21\}$ — linearly redundant with r_5/r_{21} but rescaled, giving tree splits finer-grained thresholds in the small-return region.

2.3 Feature Summary

The final feature set contains 40 engineered features across eight categories: 6 technical indicators (RSI, Bollinger Bands, MACD, PPO, MACD histogram), 7 price/moving-average spreads and crossovers, 6 multi-horizon returns, 4 multi-window realised volatility measures, 4 volume-based features (ratio, trend, log dollar

volume, and open-close return), 4 rate of change/z-score features, 2 calendar features (month and weekday), and 7 cross-sectional features (deciles plus dollar-volume rank).

3. Model Development

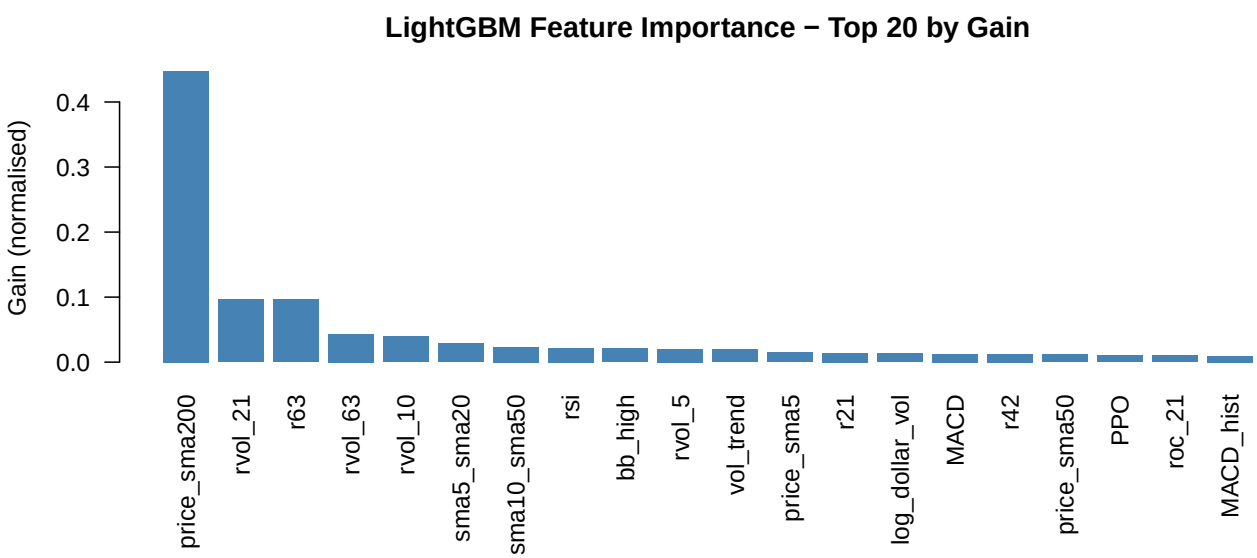
Following the Week 8 tutorial we use **LightGBM** to predict cross-sectional ETF returns. We treat the **prediction horizon** (candidate values: 5, 10, 21 days) alongside model complexity parameters as joint hyperparameters, tuning all via time-series CV with **IC** (Information Coefficient) as the evaluation metric.

No look-ahead risk from precomputation. We precomputed `feat_3d` on the full training period before constructing CV folds. This is safe because each feature `feat_3d[i, j, k]` depends only on prices and volumes up to and including day i: RSI and EMA use only recursive past-value smoothing; rolling variances use only cumulative sums of past observations; cross-sectional deciles rank the 100 ETFs within day i only. We deliberately avoid global normalisation (e.g., dividing by the period-wide standard deviation), which would genuinely leak future information.

3.1 Single Fold Evaluation

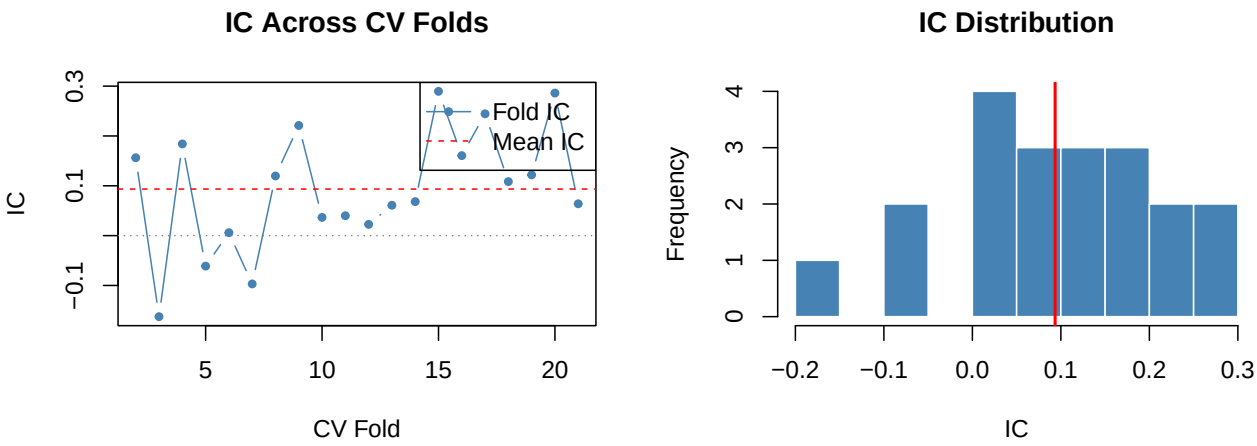
We the first CV fold with non empty validation labels (fold 1's validation forward returns extend into the test period and are all NA).

```
## Validation IC (fold 2): 0.1563
```



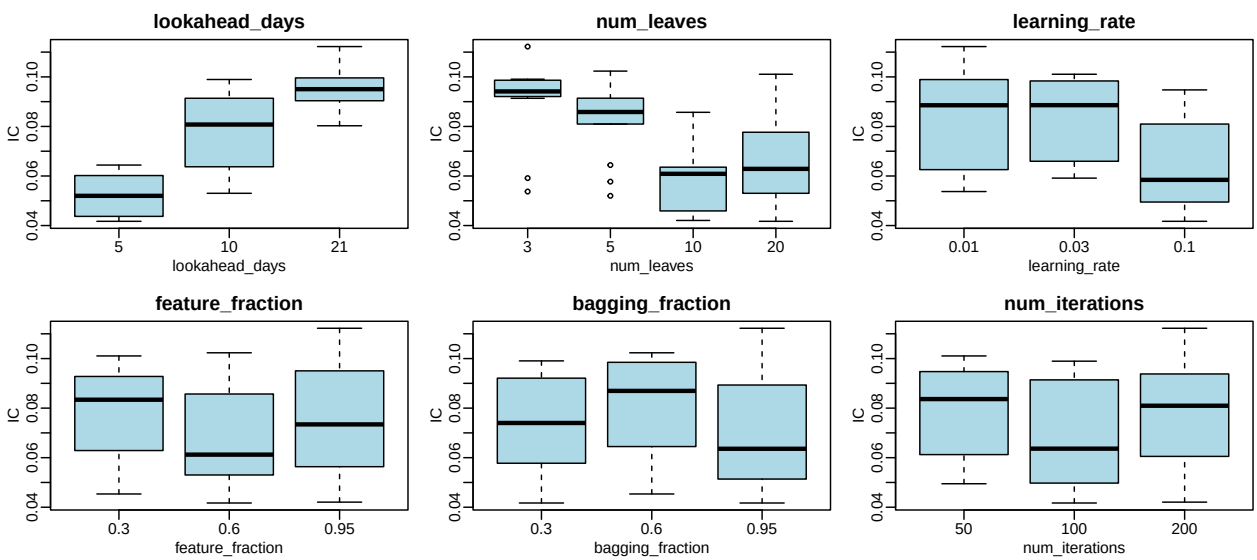
3.2 Multi-Fold Validation

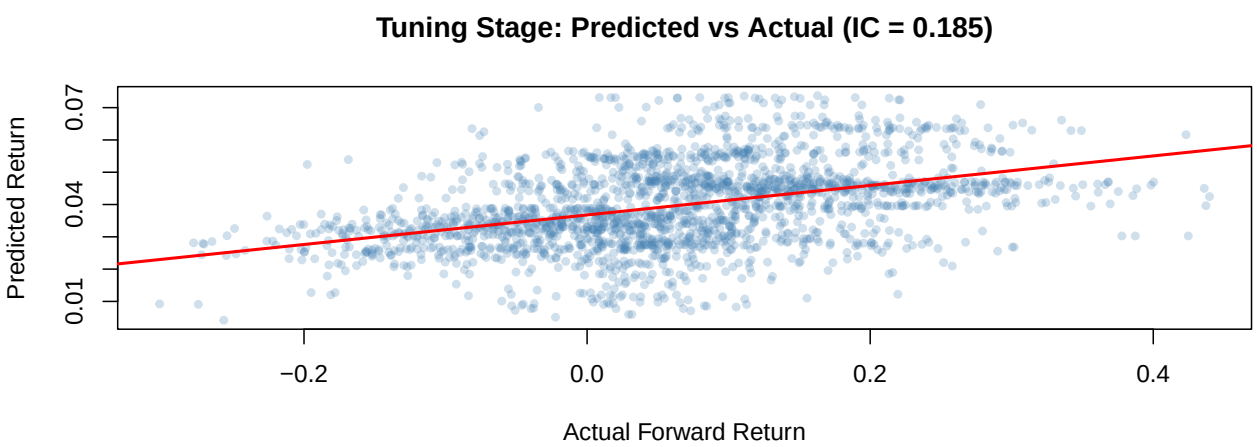
We evaluate across all available CV folds to obtain a more robust IC estimate and assess stability across different time periods.



3.3 Hyperparameter Tuning

We search jointly over **prediction horizon** (lookahead: 5, 10, 21 days) and model complexity parameters.



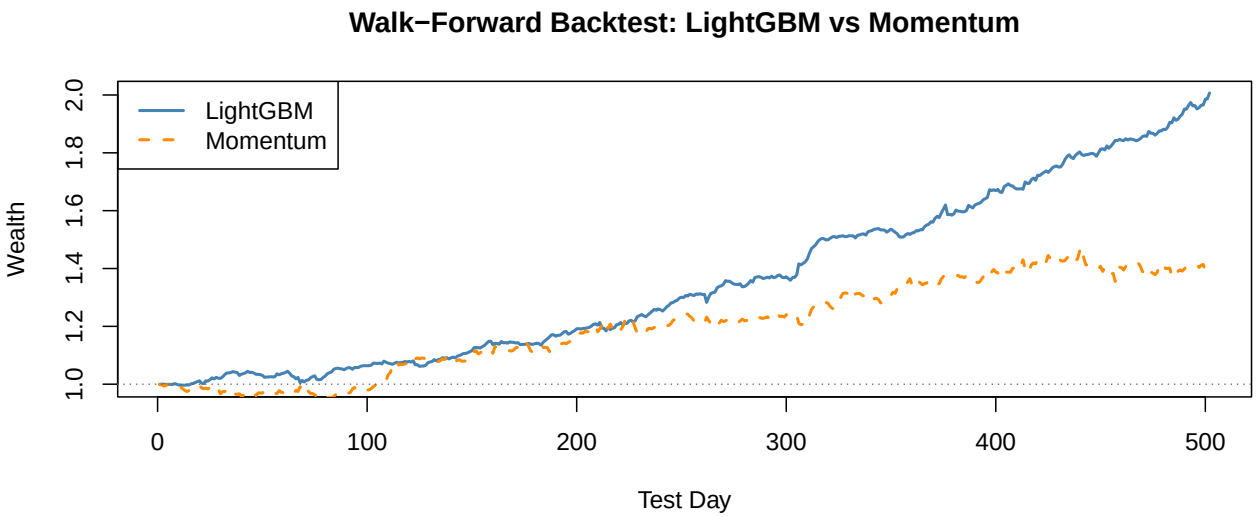


Selected hyperparameters (`lookahead = 21`, `num_leaves = 3`, `learning_rate = 0.03`, `bagging_fraction = 0.6`, `num_iterations = 200`): a 21-day horizon captures medium-term momentum; shallow trees with slow learning and strong regularisation prevent overfitting on noisy ETF returns.

3.4 Model Comparison

We compare the proposed LightGBM strategy against a simpler momentum benchmark. The two strategies are compared under exactly the same walk-forward framework: same train/test split, same daily close-price execution, same transaction cost of 5 basis points, and same portfolio size constraints. Therefore, any performance difference can be attributed to the signal-generation method rather than to differences in implementation.

```
##                    LogWealth FinalWealth SharpeRatio MaxDrawdownPct
## LightGBM Strategy     0.6966      2.0069        4.70           3.78
## Momentum Strategy     0.3359      1.3993        1.48           8.16
```



The momentum benchmark ranks ETFs by a single past-return score $s_i^{\text{mom}}(t) = r_i^{\text{past}}(t)$, implicitly assuming $\mathbb{E}[r_{i,t+1} \mid \mathcal{F}_t]$ depends mainly on this one feature. LightGBM instead models the conditional mean $\hat{r}_{i,t+1} \approx \sum_{k=1}^{K} f_k(x_{i,t})$ on a p-dimensional feature vector, so it can capture non-linearities and interactions that a one-dimensional rule cannot. When the true signal is multi-dimensional, this reduces approximation error, producing better cross-sectional rankings. The backtest confirms this: LightGBM achieves higher terminal log wealth, a substantially larger Sharpe ratio, and a smaller maximum drawdown than the momentum benchmark — consistent with a higher signal-to-noise ratio in its portfolio returns.

4. Final Strategy Explanation

4.1 Step-by-Step: How a Trade is Made

The complete trading pipeline runs inside `initialise_state` and `trading_algorithm` in `GroupA.R`. Below we trace each step explicitly using real data.

Step 1 — Initialise: Build matrices and train the LightGBM model.

`initialise_state` reshapes the long-format training data into wide price/volume matrices, calls `compute_all_features` to build `feat_3d`, then `build_training_data` to flatten it, and finally `train_lgb_fast` to fit the model.

Step 2 — Receive new data: append today's prices and drop the oldest row.

Each day `trading_algorithm` drops the oldest row from the sliding-window matrices and appends the new row, keeping the window exactly `train_length + warmup_days` rows.

Step 3 — Extract features: call `extract_today_features` for all 100 ETFs at once.

This is the same function called inside `trading_algorithm`. It computes RSI, MACD, Bollinger bands, returns, volatility, volume, and cross-sectional deciles — all O(1) using the stored EMA/RSI state and a

trailing window of close/vol/open.

Step 4 — Predict and Rank and generate trades: long top-10, short bottom-10.

Position size is divided by `lookahead` (21) because the portfolio carries 21 overlapping position layers simultaneously; dividing ensures the total notional stays proportional to current wealth.

```
## Position size per slot: 0.002381
```

```
## LONG  (top 10): ARV, DLT, DRX, DYT, HNTV, KJYC, KXRV, QVG, SWRY, VBN
```

```
## SHORT (bot 10): AMWD, CDRX, GTFX, HMY, JFK, JQR, JZKP, QVNR, RWXT, WQX
```

Step 5 — Position rolling window: insert the new position and track the 21-layer stack.

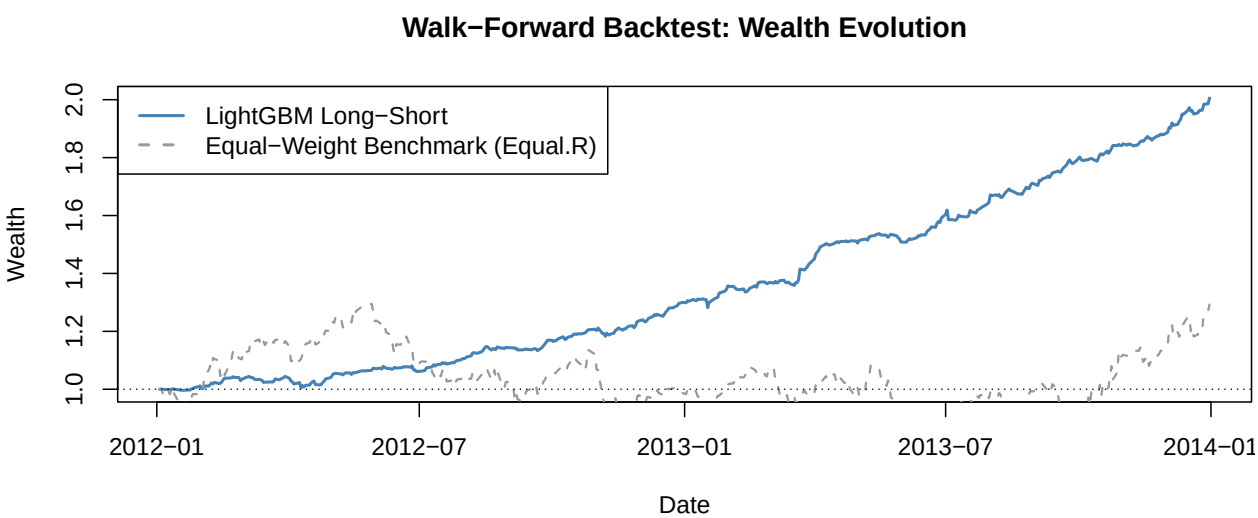
The `[21 x 100]` positions matrix updates every day with three lines identical to `trading_algorithm` in `gbm1.R`:

We replay this for 25 test days using `extract_today_features` and `predict` from `gbm1.R`, without retraining, to show how the stack builds up.

Position rolling window. The state holds a `[21 x 100]` positions matrix. Each day the new position vector is inserted at row 1 and the oldest row is dropped. Daily P&L is `sum(colSums(positions) * r1d)` — at steady state 21 overlapping layers are active, which is why `pos_size` is divided by `lookahead = 21`.

Incremental EMA/RSI updates (O(1)). Between retrains, `trading_algorithm` updates EMA12, EMA26, MACD signal, and RSI gain/loss averages with a single multiplication per symbol per indicator. Every 21 days a full batch recompute from `compute_all_features` resets the state to prevent floating-point drift.

Warmup period (200 days). SMA200 needs 200 days of history; `build_training_data` skips all rows with index ≤ 200, ensuring every training sample has valid features.

4.2 Performance Summary

We validate using the `walk_forward` function from `Test/walk_forward.R` (inlined to avoid its standalone execution block), mirroring the grader's evaluation exactly. The **equal-weight buy-and-hold** from `Equal.R` is the baseline.